

Lab 2: Maxiumum value of an array
Geavanceerde Computerarchitectuur: Labosessie

(B-KUL-JLI1QS)

Timon Coen

February 25, 2024



Group: MEIswe

Instructor: Alex Marinšek

Introduction

In this lab session, the focus lies on the way you program the GPU which can have a lot of effects its performance. In the example we're handling, we need the result of other operations or we need to read and/or write with multiple threads at once. There exist atomic operations, but in fact in stead of performing operations in parallel, we form a queue of all the operations who want to access the resource, thus handling that queue will result in a kind of for-loop behaviour.

There is an option to reduce the waiting time for the resource and that is with reduction. In this lab session we are searching for the maximum in an array. The way this can be done with reduction is by taking the first two elements and checking which one is the largest. The largest element will be placed on the first place. When doing this for the rest of the elements and repeating this but then skipping each second element and so forth, we reach a complexity of $O(log_2(N))$ in stead of the $O(N)$ with a for-loop or with the atomic operations.

Results

Three implementations were made: with a for-loop on the CPU, with `atomicMax()` on the GPU and with reduction on the GPU. As both the for-loop and `atomicMax()` have the same time complexity of $O(N)$, we'll compare them first with each other. To clearly see the pattern, we see on figure 1b the CPU execution time is linearly growing with a larger number of inputs, both for the execution with a for-loop and the execution using `atomicMax()`. We also see the `atomicMax()` is much better due to the fact that there is still paralellisation going on, but sometimes some threads have to wait. With the execution on the CPU, it is fully sequential and with atomic operations it is partially sequential as some threads can still be executed in parallel.

It is also worth looking at the execution for very small arrays. In figure 1a we can see the CPU has a very small execution time compared to the GPU with `atomicMax()`, due to the fact that is takes some time to initialise the threads on the GPU. There we again see that it only becomes interesting to use the GPU from the point we are working with a large input size.

For the reduction we expect the execution time to be a bit higher, due to the synchronisation of the threads, as in every reduction step we need to wait for the latest thread. But here again, if we're working with very large input sizes, the GPU will eventually have a lower execution time than the CPU or GPU with `atomicMax()`, due to the time complexity being $O(log_2(N))$ and the time complexity of the others being $O(N)$. Due to technical issues, I wasn't able to get correct data for the execution time of the implementation with reduction.



If the input is larger than 2048, you need to make use of multiple blocks being called from one block. Making use of recursive blocks, in the first place limiting the $2048 * 2048 = 4194304$ and in the second place to $2048 * 2048 * 2048 = 8589934592$. The recursion can be done further and further, first making sure the maximum is found for each block and then propagating the maximum further and further.



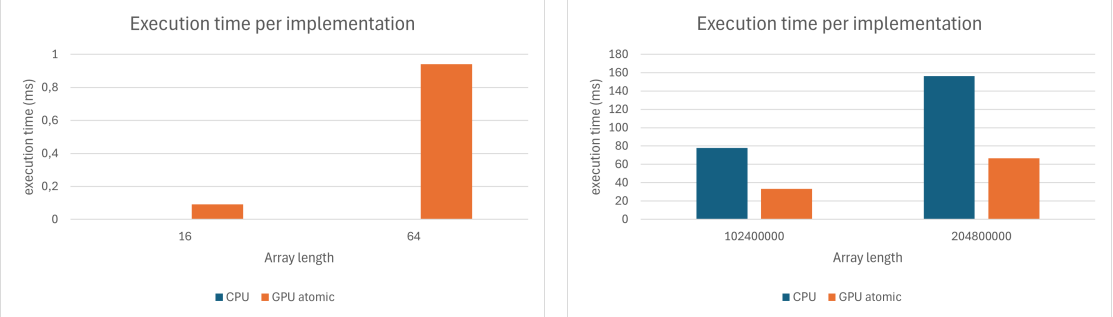
You could also work with less threads and using a for-loop for each thread to check 10 elements. Checking 10 elements in a for-loop won't take long and will initiate less threads, reducing the execution time and making the possible input size larger.



Finding the maximum in a 2D array would be the same as a 1D array, but executing every row in parallel and afterwards if every maximal value is the first element of every column, run the algorithm on the first column.

Conclusion

In some cases if you need to read/write to a resource with different treads at the same time. If that access is not managed correctly, you can run into problems. Therefore you need to use atomic operations. These can be slow as they are still $O(N)$. To improve the execution time, you can make use of reduction. Like this, there will be no need for atomic operations, making the execution faster.



(a) (b)

Figure 1: Execution time in ms for the three implementations at different input sizes